

Atelier : Modélisation Supervisée (Régression ou Classification)

Objectifs

1. **Mettre en œuvre un pipeline ML complet** : De la préparation finale des données à l'évaluation du modèle.
2. **Maîtriser la librairie `scikit-learn`** : Utiliser les fonctions clés pour l'entraînement (fit) et la prédiction (predict).
3. **Visualiser et Interpréter les performances** : Comprendre si le modèle est bon et pourquoi.

Outil	Utilisation
Environnement	Google Colaboratory (Colab)
Librairies Python	Pandas (pour les données), scikit-learn (pour la modélisation), Matplotlib/Seaborn (pour la visualisation des résultats).

Jeu de Données

Le groupe utilisera le **Notebook Python nettoyé et structuré** de l'Atelier précédent, avec sa **variable cible (Y)** et ses **variables explicatives (X)** clairement définies (ex: **Y = Consommation énergétique** ; **X = Météo, Surface, Localisation**).

Phase 1 : Préparation Finale et Séparation des Données

Étape	Consignes Python Clés	Justification
1. Encodage des Catégories	<code>pd.get_dummies()</code> ou <code>OneHotEncoder</code> de scikit-learn	Convertir les variables catégorielles (ex: <i>type de ville</i> , <i>mois</i> , <i>type de bâtiment</i>) en format numérique (colonnes 0/1) pour que le modèle puisse les interpréter.
2. Définition X et Y	<code>X = df.drop('cible', axis=1)</code> , <code>Y = df['cible']</code>	Isoler la variable à prédire ($\text{\textit{Y}}$) des variables utilisées pour la prédiction ($\text{\textit{X}}$).
3. Séparation Train/Test	<code>train_test_split(X, Y, test_size=0.3, random_state=42)</code>	Étape critique : Séparer les données en deux ensembles (70% Entraînement / 30% Test) pour s'assurer que le modèle est évalué sur des données jamais vues .
4. Mise à l'Échelle (Scaling)	<code>StandardScaler().fit_transform(X_train)</code>	Mettre les variables sur la même échelle (important pour certains modèles comme la Régression ou les Réseaux de Neurones) afin qu'aucune variable ne domine le calcul à cause de sa magnitude (ex: 10^9 pour la surface vs 20 pour la température).

Phase 2 : Modélisation et Entraînement

Choix du Modèle : Le groupe choisira un modèle en fonction de la nature de sa variable cible (Y):

Nature de Y	Type de Modèle (scikit-learn)	Rôle dans l'Atelier
Numérique Continue (Consommation, Émissions)	Régression : <code>RandomForestRegressor()</code> ou <code>XGBRegressor()</code>	Le modèle apprend à prédire une valeur (ex: 15.4 GWh).
Catégorielle (Risque Faible/ Moyen/Élevé)	Classification : <code>RandomForestClassifier()</code> ou <code>LogisticRegression()</code>	Le modèle apprend à prédire une classe (ex: Risque Élevé).

Étapes d'Entraînement :

- 1. Instanciation** : `model = RandomForestRegressor(n_estimators=100)`
- 2. Entraînement** : `model.fit(X_train, Y_train)`
- 3. Prédiction** : `Y_pred = model.predict(X_test)` (Appliquer le modèle entraîné aux données de test).

Phase 3 : Évaluation des Performances et Interprétation

Tâche	Métriques d'Évaluation (scikit-learn)	Visualisation
Régression	R^2 Score (R2, à quel point le modèle correspond aux données) et $RMSE$ (Erreur quadratique moyenne, l'erreur moyenne de prédiction).	Nuage de Points : Comparer les Y_{vrais} vs $Y_{\text{prédits}}$ (les points doivent s'aligner sur une diagonale).
Classification	Accuracy (Précision globale) et F1-Score (équilibre Précision/Rappel).	Matrice de Confusion (confusion_matrix) : Visualiser où le modèle fait des erreurs (ex: confondre "Faible Risque" et "Moyen Risque").

Interprétation Critique :

- Feature Importance** : Utiliser la fonction `.feature_importances_` du `RandomForest` pour visualiser (diagramme en barres) quelles sont les **variables qui ont le plus influencé la prédiction** (ex: la température est-elle plus importante que l'humidité pour les incendies ?).